

CODSOFT CYBER SECURITY INTERNSHIP

Task 3: Secure Code Assessment

Submitted By: Simran Teja

Technology Used: Python

1. Project Objective

The objective of this project is to analyze a vulnerable login system, identify common security issues, and improve the application by applying secure coding practices. The project demonstrates the difference between an insecure login system and a secure login system using password hashing with Python.

2. Identified Vulnerabilities

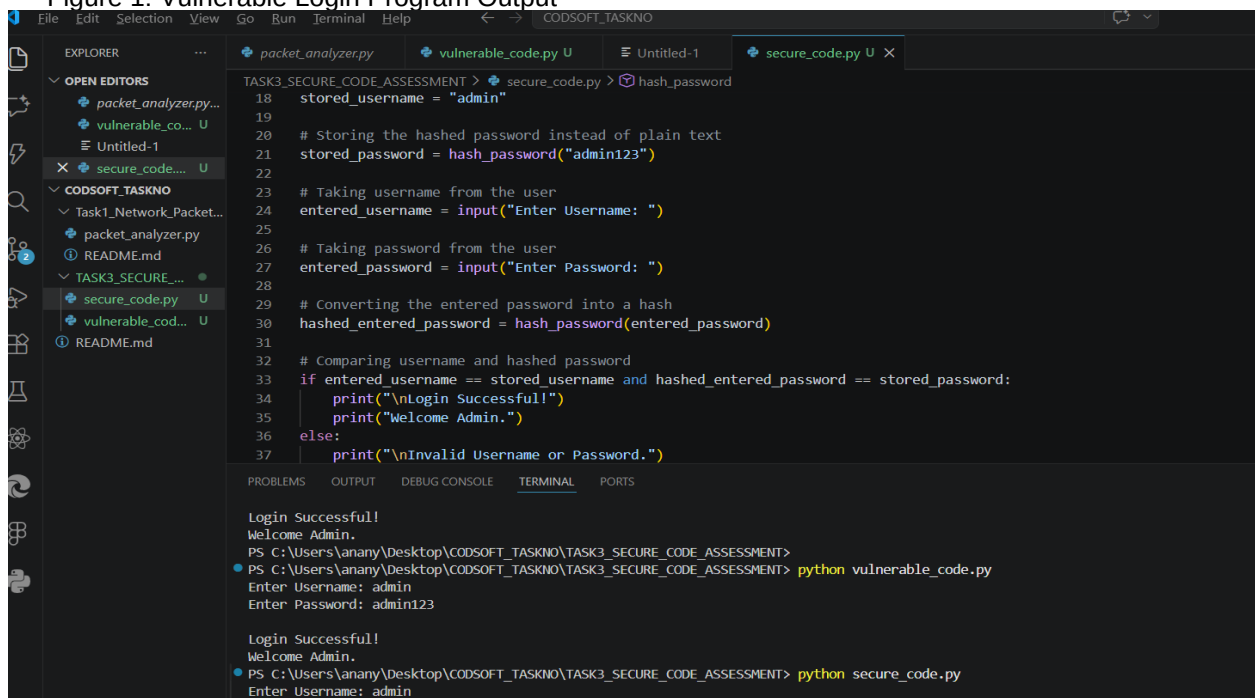
- Username and password were hardcoded directly in the source code.
- Password was stored and compared in plain text.
- Sensitive information could be exposed if the source code was accessed.
- The login process did not follow secure coding practices.

3. Security Improvements

- Implemented SHA-256 password hashing using Python's hashlib library.
- Replaced plain text password comparison with hash comparison.
- Avoided direct password storage.
- Added comments for better readability.

4. Project Output

Figure 1: Vulnerable Login Program Output

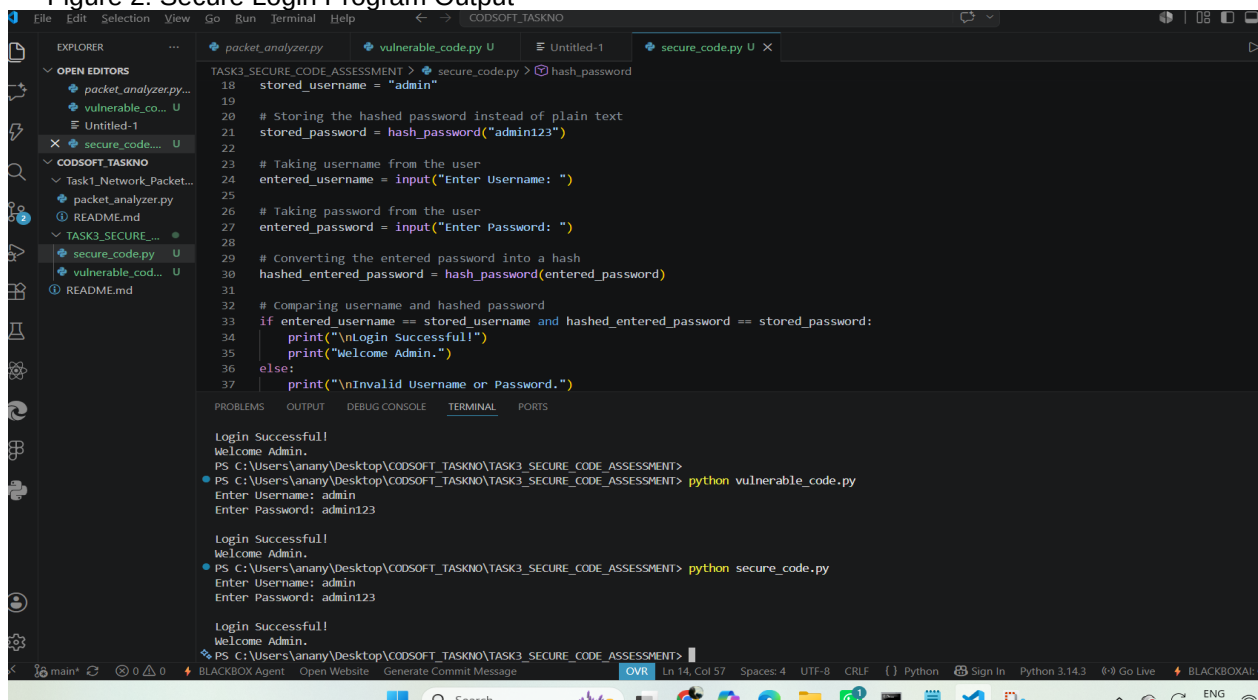


The screenshot displays a code editor with the following components:

- EXPLORER:** Shows the project structure with files like `packet_analyzer.py`, `vulnerable_code.py`, and `secure_code.py`.
- EDITOR:** Displays the `secure_code.py` file with the following Python code:

```
18 stored_username = "admin"
19
20 # Storing the hashed password instead of plain text
21 stored_password = hash_password("admin123")
22
23 # Taking username from the user
24 entered_username = input("Enter Username: ")
25
26 # Taking password from the user
27 entered_password = input("Enter Password: ")
28
29 # Converting the entered password into a hash
30 hashed_entered_password = hash_password(entered_password)
31
32 # Comparing username and hashed password
33 if entered_username == stored_username and hashed_entered_password == stored_password:
34     print("\nLogin Successful!")
35     print("Welcome Admin.")
36 else:
37     print("\nInvalid Username or Password.")
```
- TERMINAL:** Shows the execution of the program. It runs `python vulnerable_code.py` and `python secure_code.py`. Both executions result in "Login Successful! Welcome Admin." when the username is "admin" and the password is "admin123".

Figure 2: Secure Login Program Output



The screenshot displays a Visual Studio Code editor window with the following components:

- EXPLORER:** Shows the project structure with files like `packet_analyzer.py`, `vulnerable_code.py`, and `secure_code.py`.
- EDITOR:** Displays the `secure_code.py` file with the following Python code:

```
18 stored_username = "admin"
19
20 # Storing the hashed password instead of plain text
21 stored_password = hash_password("admin123")
22
23 # Taking username from the user
24 entered_username = input("Enter Username: ")
25
26 # Taking password from the user
27 entered_password = input("Enter Password: ")
28
29 # Converting the entered password into a hash
30 hashed_entered_password = hash_password(entered_password)
31
32 # Comparing username and hashed password
33 if entered_username == stored_username and hashed_entered_password == stored_password:
34     print("\nLogin Successfull")
35     print("Welcome Admin.")
36 else:
37     print("\nInvalid Username or Password.")
```
- TERMINAL:** Shows the execution of the program. It runs `python vulnerable_code.py` and `python secure_code.py`, both of which prompt for a username and password. In both cases, the user enters "admin" and "admin123", resulting in the output: "Login Successfull" and "Welcome Admin."

5. Conclusion

This project demonstrates how a vulnerable login system can be improved using secure coding practices. Password hashing protects user credentials and improves application security.